Lugworm

Complete Reference

Jasper Vollmert
Institute of Digital and Autonomous Construction (IDAC)
Hamburg University of Technology
jasper.vollmert@tuhh.de

Generated from `Lugworm` source (Components/*.cs, Core/*, docs/DECISION_LOG.md): September 22, 2026

Abstract

This document merges the quick-lookup manual and the full technical reference for the **Lugworm** Grasshopper plugin (assembly `Lugworm`, author `Lugworm`) into a single view: every component below carries its compact parameter table immediately followed by the technical detail behind it, so there is one place to look rather than two documents to cross-reference. For a plain-language introduction see *beginner-guide.md*; for guided practice see *exercises.md*. This document assumes Grasshopper and G-code familiarity.

The plugin documented here is the current, actively developed C# tool. An earlier Python GHPython prototype (`ClaySlicer2`/`ClayMover2`/`ClayGcode2`, `clay_core_v2.py`) is superseded: per `docs/DECISION_LOG.md` entry D030, that Python toolkit is kept only as a fixture-regeneration script for the test suite of this plugin, not as a user-facing tool.

Contents

1 Overview and pipeline **3**
1.1 Component index . . . 3

2 Architecture and compute model **4**
2.1 Heavy components and the compute gate . . . 4
2.2 Document units, language, and file I/O . . . 4
2.3 Multi-geometry input . . . 5
2.4 Geometry validation policy . . . 5

3 Slice: Panel 1 Slice [Heavy] **5**

4 Place: Panel 2 Place **6**
4.1 Print Bed . . . 6

5 G-code Generator: Panel 3 G-code [Heavy] **6**
5.1 Core parameters . . . 6
5.2 Optional toggle groups . . . 8
5.3 G-code Restart . . . 8
5.4 Save G-code . . . 9

6 Robot selection and simulation: Panel 4 Robot **9**
6.1 Robot Definition . . . 9
6.2 Robot Simulation . . . 10
6.3 SCARA profile: “Potterbot Scara v4” . . . 10

7 Analyze components: Panel 5 Analyze **10**
7.1 Overhang **[Heavy]** . . . 10
7.2 Print Stats . . . 11

8 Calibration workflow: Panel 6 Calibrate **12**
8.1 DoE Design . . . 12
8.2 Printing and measuring the runs . . . 12
8.3 RSM Evaluate **[Heavy]** . . . 13
8.4 Desirability **[Heavy]** . . . 13

9 Utility components: Panel 1 Slice **14**
9.1 Graph Mapper . . . 14

10 Rationale: pressure and pump **14**
10.1 Pump extrusion (MAI 2PUMP PICTOR-3D) . . . 14
10.2 Legacy calibration values (30 mm ram machine) . . . 15
10.3 A real historical bug worth knowing about (ArcWelder) . . . 15

11 Known limits and explicit placeholders **16**

12 Troubleshooting **16**

13 Testing and provenance **17**

1 Overview and pipeline

All components live under one Grasshopper tab, **Lugworm**, split into seven numbered panels that match the order you would actually use them in:

```
1 Slice | 2 Place | 3 G-code | 4 Robot | 5 Analyze | 6 Calibrate | 7
                                        Print
```

The core print pipeline:

```
your 3D geometry
      |
      v
+-------------+  contours (one    +-------------+  moved contours   +--------------+
|Slice |- branch/layer) ->|Place |------------------>|Lugworm G-code |--> .gcode
+-------------+                   +-------------+                   |   Generator  |
 "cut it into                      "put it on                       +--------------+
  layers"                           the bed,                         "write the machine
                                     fix the seam"                    instructions"
```

From `G-code Generator`, the file either goes straight to a Marlin/Duet printer, or through `G-code Restart` / `Save G-code`. The machine itself is an explicit selection: `Robot Definition` emits a profile string (`iamlab-robot:v1:...`) that every robot-dependent component accepts on one `robot` input, and `Robot Simulation` validates the program on the selected machine (see §6). Everything in the core pipeline also runs with **no robot selected at all**.

Three further component groups sit alongside the pipeline: **Analyze** (`Overhang`, `Print Stats`) for checking a slice or finished G-code before committing to print; **Calibrate** (`DoE Design`, `Lugworm RSM Evaluate`, `Desirability`, `Test Geometry`) for finding good settings for a new material via designed experiments rather than guesswork; and **Print** (the `Lugworm Duet ...` components) for direct Duet WiFi machine control, documented in *DUET_CONTROL.md* rather than here.

1.1 Component index

Display name	Nickname	Panel	Purpose
`Slice`	Slice	1 Slice	Mesh/Brep/Surface → contour tree
`Graph Mapper`	Graph Map	1 Slice	Remap numbers through an editable curve
`Place`	Place	2 Place	Bed placement and seam control
`Print Bed`	Bed	2 Place	One bed definition for Place/Simulation/Live
`G-code Generator`	G-code	3 G-code	Contour tree → Marlin/Duet G-code
`G-code Restart`	Restart	3 G-code	Resume a failed print mid-way
`Save G-code`	Save	3 G-code	Write G-code to a timestamped file
`Robot Definition`	Robot Def	4 Robot	Select the machine profile (one wire)
`Robot Simulation`	Robot Sim	4 Robot	Visualize/simulate/validate on the selected machine
`Overhang`	Overhang	5 Analyze	Per-segment overhang/printability analysis
`Print Stats`	Stats	5 Analyze	Length/volume/mass/time summary (robot-aware)
`DoE Design`	DoE	6 Calibrate	Design-of-experiments run plan generator
`RSM Evaluate`	RSM	6 Calibrate	Response-surface model fitting
`Desirability`	Desire	6 Calibrate	Recommend settings from fitted models

Display name	Nickname	Panel	Purpose
`Test Geometry`	TestGeo	6 Calibrate	Parametric calibration coupons
`Duet Connect/Status/Jog/Print/Live/Files/Terminal`		7 Print	Direct Duet WiFi machine control (see *DUET_CONTROL.md*)

The pre-2.0 `Robot` component is hidden from the ribbon but still deserializes and solves in saved documents; `Lugworm Robot Simulation` replaces it (§6).

[Heavy] marks components with a manual compute button (cached per component instance) rather than automatic recompute-on-change: `Lugworm Slice`, `G-code Generator`, `G-code Restart`, `Overhang`, `RSM Evaluate`, and `Lugworm Desirability`.

2 Architecture and compute model

`Lugworm/Components/*.cs` hold thin Grasshopper adapters (parameter registration, caching, viewport preview); `Lugworm.Core/*` holds the actual algorithms, organised by concern: `Analysis`, `Arcs`, `Clipping`, `Doe`, `Duet`, `Extrusion`, `Gcode`, `Geometry`, `Machine`, `Paths`, `Robot`, `Slicing`. This mirrors the Python prototype's split (thin adapter + pure core) but as a compiled library with a dedicated test project (`Lugworm.Core.Tests`, 980+ tests as of the 2.6.0 release).

2.1 Heavy components and the compute gate

[Heavy] components show a manual compute button instead of recomputing on every canvas edit. Results are cached per component instance (`SolveCache`, keyed by instance GUID) and evicted on component deletion or document close. The robot simulator's cache key is a content fingerprint (order-sensitive FNV-1a hash over every sampled coordinate, extrude flags, and the full G-code string: a reordering or interior-vertex edit is guaranteed to invalidate it, fixed 2026-07-14 after an earlier bounding-box-sum fingerprint let stale `danger=false` results survive a real edit). `Lugworm Save G-code` and `G-code Restart` are not Heavy (cheap/live), though `Save G-code` has its own manual `write` gate for a different reason: avoiding re-writing a file on every recompute.

2.2 Document units, language, and file I/O

Version 2.6.0 introduced three behaviours that apply across the plugin:

- **Document units.** Geometry is converted at the Rhino/Core boundary, so a document in millimetres, inches, or metres slices correctly and previews overlay the document geometry at document scale. Plain-number inputs documented in mm (`layerHeight`, `width`, `spacing`, tolerances, ...) remain physical millimetres regardless of document units, and emitted G-code / Duet traffic is always plain mm. Changing document units mid-session invalidates Heavy caches ("no cached result: press Compute") rather than replaying a stale-scale result.
- **Language/culture.** Machine-readable text (G-code, CSV, reports) is culture-invariant: a German-locale Windows emits decimal points, never commas. CSV *inputs* (RSM tables, pump calibration) are detected per file as English (`1.5`) or German (`1,5`); a file mixing both conventions, using thousands separators, or with malformed quoting is refused with the offending line named, never silently misread.
- **Background file I/O.** File reads and writes (G-code save/restart, pump CSV, overhang CSV, RSM read/export) run in cancellable background workers so the canvas never blocks on a slow disk or network share. While a job is pending the component replays its last good outputs and reports "writing/loading ..."; the result lands on the follow-up solve.

2.3 Multi-geometry input

`Slice`'s `geo` is a list parameter (single items auto-promote). A single object emits the old flat `{i}` path structure unchanged (backward compatible); a true list emits `{object;layer}` paths. Downstream components (`Place`, `G-code Generator`, `Robot Simulation`) iterate branches generically and handle both shapes. Multi-object printing is **sequential** (all of object A's layers, then object B's, ...), order given by the input list or overridden by the `printOrder` toggle-group input: deliberately not layer-interleaved, since interleaving invites SCARA nozzle/part collisions and travel time would dominate anyway; true interleaving needs a collision-aware travel planner that does not exist yet.

2.4 Geometry validation policy

From `Core/Geometry` (`GeometryInput.Prepare` + `MeshChecks.Analyze`):

- **Rejected outright:** null/unsupported type, unmeshable Brep, zero faces, non-manifold edges.
- **Warned, still sliced:** open mesh (naked edges), degenerate faces, disconnected components, non-solid Brep.
- **Auto-fixed, reported:** inconsistent face normals via `Mesh.UnifyNormals`.

Breps still section via precise `BrepPlane` intersection (unchanged from before), but are *additionally* meshed for validation/adaptive-slope/conical-shear use: a Brep pays a meshing cost even in plain planar mode now, with no size-based opt-out currently built.

3 Slice: Panel 1 Slice [Heavy]

"Slice a mesh/brep/surface (or a list of them) into a clay contour tree."

Parameter	Unit	Default	Description
`geo`	Geometry, list	required	Mesh/Brep/Surface(s) to slice; validated per §2.3.
`layerHeight`	mm	3.0	Layer spacing.
`firstLayer`	mm	3.0	`0` → half a normal layer.
`wallCount`	int	1	Number of perimeters.
`spacing`	mm	10.0	Wall/infill spacing: **hard error if** ≤ 0.
`mode`	int	0	0 Planar / 1 Conical.
`infill`	int	0	0 None / 1 Zigzag / 2 Concentric / 3 Spiral / 4 Fermat.
`cornerRadius`	mm	0.0	Fillets sharp corners (over-fill correction).
`tol`	mm	0.05	Linearisation/simplification tolerance.

Toggle groups: Adaptive (`hMin` = layerHeight×0.5, `hMax` = layerHeight×1.5, planar-only); **Conical** (`coneAngle` = 20°, `nozzleClear` = 25°, advisory collision warning only); **Zigzag** (`angle` = 0°); **Spiralize** (menu toggle, no parameter: requires exactly one closed contour per layer, all layers); **Meshing** (`meshDensity` = 0.5, `meshMaxEdge` = 0, `meshMinEdge` = 0: controls Brep-to-mesh conversion quality for validation/adaptive/conical); **Multi-object** (`printOrder`, an index list).

Outputs: `contours` (Curve tree), `layers` (int), `info` (str: reports per-object issues; one bad object in a list does not abort the whole solve).

Non-planar specifics. Because conical ring spacing follows the sheared surface rather than a constant vertical Z step, **extrusion volume in conical (and adaptive) mode is only nominal**

unless the `varh` group of the G-code component is enabled (§5.1). The heat method (`mode = 2`) was **retired in 4.0.0**: it emitted perimeter rings only and fell back to planar on most real meshes. A saved definition carrying `mode = 2` slices planar and reports a warning.

4 Place: Panel 2 Place

"Place contours on the bed and set the seam (clay / LDM)." Not Heavy (live).

Parameter	Unit	Default	Description
`contours`	Curve tree	required	From Slice.
`bed`	Bed definition		Optional `Print Bed` wire; supplies the print centre when `printCenter` is unwired.
`printCenter`	Point, mm	(339, −193, 0)	The lab's SCARA bed "sweet spot"; wins over the bed definition when wired.
`zOffset`	mm	0.0	Added after placement.
`move`	Vector	zero	Extra manual shift alongside centring.
`seamMode`	int	1	0 Keep / 1 Align / 2 Stagger / 3 Nearest / 4 Guide / 5 Track-below.

Seam group: `seamAngle` (0°, modes 1 to 2), `seamPoint` (mode 3), `seamCurve` (modes 4 to 5: "Guide" follows a supplied curve, "Track-below" keeps the seam aligned with the layer underneath rather than a fixed target: new in the 2026-07-14 run).

Placement math: $dx = printCenter.X - bboxCenter.X + move.X$ (and analogously for Y, and Z with `zOffset`).

Outputs: `moved`, `seams` (point tree), `info`. Placement is robot-independent: whether the placed part is reachable is validated in one place only, `Robot Simulation` (§6), so Place itself emits no reach warning.

4.1 Print Bed

"Define the print bed once: size, position and default print center: and wire it to placement, simulation and live view." Not Heavy. Emits a versioned definition string (`iamlab-bed:v1:w=...;d=...;cx=...`) consumed by `Place`, `Robot Simulation` and `Lugworm Live View`; an unrecognised string fails loudly rather than resolving to wrong geometry.

Two separations are deliberate. **Plate centre is not print centre**: `center` is the plate, `printCenter` is where a part's bottom-centre lands: (468, −441) versus (339, −193) on the lab machine. And **reach belongs to the arm, not the plate**: `reachMin` and `reachMax` of the bed default to "ask the robot profile" and exist only so a physically restricted plate can narrow the annulus. Every pre-existing bed input on the consuming components survives and overrides the definition when wired, so saved documents are untouched.

5 G-code Generator: Panel 3 G-code [Heavy]

"Convert any curve/polyline tree into material-extrusion G-code (Marlin / Duet RRF)."

5.1 Core parameters

Parameter	Unit	Default	Description
`crvs`	Curve tree	required	From Place.
`robot`	Robot definition		Selects the coordinate frame (and machine defaults). **Unwired = legacy Potterbot frame**; see below.
`layerHeight`	mm	must match Slice	E cross-section height.
`width`	mm	10.0	Layer width.
`ramDiameter`	mm	94.0	Cartridge bore; warns (not a hard limit) outside 5 to 200 mm.
`pressureCoeff`		1.2	Overall flow multiplier, final calibration knob.
`printSpeed`	mm/s	43	Extruding feedrate.
`travelSpeed`	mm/s	30	Non-extruding feedrate.
`retractSpeed`	mm/s	1	Ram screw speed for retract & prime (the screw moves in raw mm; retraction ≈ 0.1 mm, so keep this small).
`firstLayerSpeedFactor`		0.5	Speed multiplier, first layer only.
`retraction`	0/1		Master on/off for pull-back/prime behaviour.
`seamTol`	mm		Seam-adjacent tolerance for coast/lead-in placement.
`firmware`	enum	Duet-RRF / Marlin	Changes exact G-code dialect emitted.
`relativeE`	bool	true	`M83`-style relative E.
`pressureOffset`	mm		Plunger pull-back ("retraction" for paste).
`primeExtra`	mm		Extra prime on restart.
`coast`	mm		Stop extruding this far before a path end.
`zHop`	mm		Travel lift.
`hopSpeed`	mm/s		Z-hop feedrate.
`footerZLift`	mm		End-of-print lift.
`dwell`	ms		Pause after prime.
`leadIn`	mm		Flow ramp-in length.
`leadInBoost`			Flow multiplier during lead-in.
`minLayerTime`	s		Slow a layer that would print faster than this.
`zOffset`	mm	0.0	Nozzle-to-bed calibration shim, added on top of geometry the slicer has already placed at its true height. The slicer owns layer height; leave at 0 unless the Z zero of the machine needs a correction.
`tol`	mm		Curve simplification tolerance.
`savePath`	path, optional		Write directly from this component (background write; the saved path is reported on the completion re-solve).

Coordinate frame. `GcodeConfig.CoordinateFrame` comes from the wired robot definition and is applied in the emitters (arc sense flips with the reflection). An **unwired** `robot` input resolves to `potterbot-scara-v4` and its calibrated legacy frame (a mirror about $y = -x$): every G-code emitted before 2.0.0 carried that reflection, so defaulting to identity would have silently mirrored the lab's saved definitions. A Cartesian user must wire a definition (`generic-cartesian`, `generic-duet-cartesian`, or `none`) to get the identity frame. The active frame is reported in `info` (`frame:...`).

5.2 Optional toggle groups

- **advance / retractaccel**: firmware pressure-advance and acceleration overrides (the `M900`/`M204`-equivalent hand-off to firmware).
- **arcs**: built-in arc fitting (`G2`/`G3`) instead of linear-only output.
- **arcwelder**: post-process via the ArcWelder CLI instead of the built-in fitter; **depends on** `arcs` (auto-disabled if arcs is off). See §8.3 for a critical historical bug here.
- **skirt**: purge ring before the real print.
- **floors**: solid base layer count.
- **headeroverride**: replace the auto-generated header entirely.
- **pump**: MAI 2PUMP PICTOR-3D support: `nozzleDiameter`, `pumpCalibCsv`, `slipFactor`. See §8.1: **this group only calculates a recommended pump setting; it does not alter E-values in the emitted G-code.**
- **varh** (variable layer-height flow): `varhMin` (0.25), `varhMax` (2.5): clamps the per-point flow multiplier derived from `VariableFlow.LocalHeights`. **Turn this on whenever slicing used adaptive or conical mode.**
- **ecomp** (slope-compensated flow): `eCompRefMul` (1.0 = no-op), `eCompRefDeg` (45°): experimental extra-flow correction on inclined walls via `SlopeFlow.Multipliers`. **NEEDS_INPUT: uncalibrated; leave off until an inclined-wall calibration print has been run.**

Outputs: `gcode` (list), `gcode_text` (str), `preview`/`travels` (curve trees), `length` (mm), `volume` (cm^3), `time` (min), `info`, `pump` (text summary, populated only if the pump group is active), `start` (first point), `startGcode` (a jog-to-start block, useful for manually positioning before a resumed/robot print).

5.3 G-code Restart

"Resume a failed print mid-way, cutting the G-code at a Z height, Duet % or XYZ position and rebuild it from there." Heavy.

Parameter	Unit	Description
`gcode`	Text list	From the G-code component.
`z` / `percent` / `point`	mixed	Mutually exclusive cut selectors; priority **point** > **percent** > **z** if more than one is supplied. `point` is a Rhino document-coordinate XYZ point (for example, a Live or Console position output), not raw Duet machine XYZ.
`primeE`	mm	Re-prime distance at the resume point.

Parameter	Unit	Description
`travelSpeed`, `hopSpeed`, `retractSpeed`	mm/s	Resume-move feedrates.
`clearance`	mm	Safety lift before the resume travel.
`savePath`	path	Output location.

Outputs: `gcode`, `gcode_text`, `resume` (the point execution resumes from), `printed`/`remaining` (curve lists: useful to visually confirm the cut point before committing), `info`.

5.4 Save G-code

"Write G-code to a folder with a date/time-stamped filename."

Parameter	Unit	Description
`gcode`	Text list	Input G-code.
`folder`	path	Destination folder.
`prefix`	str	Default `"print"`.
`write`	bool	Wire a **Button**, not a persistent Toggle, so it fires once per click rather than on every solve.
`dateTime` / `format`		Override group; `format` default `"yyyy-MM-dd_HH-mm"`.

Outputs: `path`, `written` (bool: true only on the solve where the write completed, not persistently), `info`. The write itself runs in a background worker (2.6.0): the press reports "writing ..." immediately and `written = true` arrives on the completion re-solve, so a slow network share never freezes the canvas. The component persists `_lastPath`/`_lastInfo` across sessions so re-opening the file still shows the last write's outcome.

6 Robot selection and simulation: Panel 4 Robot

6.1 Robot Definition

"Select a robot or Cartesian machine profile for kinematics, visualization and program generation." Not Heavy. Emits a stable, versioned string, `iamlab-robot:v1:<profile-id>`, which every robot-dependent component accepts on a single `robot` input. `RobotCatalog.Resolve` also accepts canonical ids and short aliases (`"scara"`, `"none"`), so a plain text panel works as well as the component. An unknown definition raises a clear error instead of silently resolving to the wrong machine.

Profile id	Mechanism	Program	Direct control
`none`	none	RepRap G-code	
`generic-cartesian`	Cartesian	RepRap G-code	
`generic-duet-cartesian`	Cartesian	RepRap G-code	Duet (7 Print)
`potterbot-scara-v4`	SCARA	RepRap G-code	Duet (7 Print)

Adding a robot is one `RobotCatalog` entry, plus a solver in `Core/Robot/` only if it needs IK, plus a mesh provider only if it should be drawn (*ARCHI.md* §12.1). A Cartesian machine needs neither.

6.2 Robot Simulation

"Visualize G-code/toolpaths and, when a Robot Definition supplies kinematics, simulate and validate the selected robot." Replaces the pre-2.0 `Robot` component (hidden but still solving in saved documents). The numerical solvers were not changed by the unification: the dispatch on the wired profile selects which validator runs.

Parameter	Unit	Default	Description
`robot`	definition		Empty/`none` = model-free toolpath and extrusion viewer.
`gcode` / `toolpath`		required	Program (one string or lines), or curves used when G-code is empty.
`sim`	0 to 1	1	Scrub through the accumulated print.
`showRobot` / `showExtrusion`	bool	true	Viewport preview toggles.
`width` / `height`	mm	10 / 0	Layer preview cross-section (`0` = infer from layer metadata).
`bed`	definition		Optional `Print Bed`; the explicit bed inputs override it when wired.

Groups: **bed** (`bedWidth` 1000, `bedDepth` 700, `bedCenter`: visualization extents); **robotopts** (`baseOffset`: bed/toolpath origin in robot-base coordinates; `armMode`: SCARA arm mode used when the path is not anchored by homing: a path with a `G28` takes its arm mode from the homing endstops of the machine).

Outputs: `joints` (Number tree, one branch per joint: empty without kinematics), `arm`/`tcp`/`bed` (preview geometry), `reachOK`, `problems` (flagged target positions), `messages`, `info`. Any target outside the limits of the selected machine also raises a hard red runtime **Error** ("targets are outside robot limits"). **Never bypass or silence that error**: if it fires, the fix is to change the print (geometry, placement, or speeds), not to hide the message. Deposition and travel draw in different colours (RoyalBlue layer vs. thinner ochre travel), in both display modes.

6.3 SCARA profile: "Potterbot Scara v4"

The SCARA validator walks the program with `ScaraKinematics`/`RobotChecks` against the shipped Potterbot profile: link lengths $L1 = 549.39\,\text{mm}$ / $L2 = 458.62\,\text{mm}$ (firmware `M669 P/D`), reach annulus 235 mm to $L1{+}L2$, z range 0 to 1143 mm, `maxFeed` 150 mm/s, `maxJointSpeed` 120°/s, joint ranges $J1 = -311.1°$ to 49.1°, $J3 = \pm 151.93°$ (the *relative* elbow angle: firmware naming), $J2 = 0$ to 1030 mm (the prismatic Z-carriage), and a near-singularity jerk flag. It also models the real Cartesian workspace and emulates the arm-mode of the firmware and joint-wrap choices, so a toolpath that validates is one the machine will follow.

J1's firmware range spans 360.2° but wraps at the ±180° boundary: angle comparisons for range-checking accept ±360° equivalents (fixed 2026-07-14 after a real −200° pose, in range, normalised to +160° and incorrectly flagged as unsafe).

7 Analyze components: Panel 5 Analyze

7.1 Overhang [Heavy]

"Per-layer / per-segment overhang analysis for printability assessment."

Parameter	Unit	Default	Description
`contours`	tree	required	From Slice or Place.
`warnDeg`	°	25	Warning threshold: **placeholder, not calibrated**.
`failDeg`	°	45	Failure threshold, clamped ≥ `warnDeg`; **placeholder**.
`tol`	mm	0.5	Segment tolerance.
`colorMode`		0	0 = three classes, 1 = continuous magma ramp from 0 to `failDeg`.
`global`		off	Off = local reference (angle from the layer normal, the stagger); on = global reference (angle from world Z, the lean against gravity). Picks which reading the thresholds, colours and legend use.
`csvPath`	path		Export group.

Outputs: `angles` (the **primary** metric: each segment's angle vs. the *local* layer-surface normal, estimated via a windowed Newell-normal fit over an approximately 15-point window around the segment; for planar layers this is only the world-Z angle, but for non-planar (conical) layers it measures actual layer-on-layer stagger, which is what limits printability there), `anglesWorld` (the gravity-relative angle vs world Z: switch `global` on to classify by it if gravity load is considered the dominant failure mode for a given mix rather than stagger), `summary` (CSV text), `info`, `legend` (a wire for Legend) and `colors`. Colour-coded viewport preview in magma hues, three classes or a continuous ramp.

`warnDeg`/`failDeg` **are explicitly uncalibrated placeholders**: published 3D-concrete-printing studies put safe unsupported wall inclination around 15 to 35°, and the shipped defaults sit inside that envelope, but they are not measured values for any specific material. Calibrate via a staircase-cone test print and set the measured angles as plain inputs (they are not constants).

7.2 Print Stats

"Lengths, extruded volume/mass, time estimate and per-layer breakdown of a print job." Not Heavy.

Parameter	Unit	Default	Description
`gcode` / `toolpath`		required	Source.
`robot`	definition		Optional; enables the joint-speed-aware time model (see below).
`width`	mm	4	Layer width, for the geometric volume fallback.
`height`	mm	0	`0` = auto-derive from Z-steps.
`density`	g/cm^3	2.0	Material density (clay ≈ 2.0).
`ramDiameter`	mm	94	**If set and the G-code has E values, volume is computed exactly from E**; otherwise falls back to the geometric width×height estimate.
`printFeed` / `travelFeed`	mm/s	25 / 60	Feeds group.
`accel`	mm/s^2	0 (off)	Non-zero enables a trapezoidal accel-aware time model instead of the naive distance/feed estimate.

Outputs: `report` (formatted text), `timeMin`, `volumeCm3`, `massG`, `extrudeLen`, `travelLen`, `layers`, `perLayer` (tree of `[z, length, time]` per layer).

Robot-aware time (2.1.0). With a kinematic `robot` definition wired, each segment's feed is additionally capped by the joint-speed limits of the machine, inverting the same check `RobotChecks` enforces:

$$feed \leq \min(\text{requested},\ MaxFeed,\ MaxJointSpeed \cdot dist/|\Delta J_i|)$$

Near the reach boundary or the dead zone a small Cartesian step demands a large joint move, the cap collapses, and the segment correctly takes longer. The joint-limited time is reported *alongside* the Cartesian estimate with a percentage delta, and `timeMin` publishes the most constrained model available (robot-limited, else accel-aware, else the naive lower bound). `robot = none` and non-kinematic profiles leave every existing path bit-identical.

8 Calibration workflow: Panel 6 Calibrate

A structured way to find good print-parameter settings for a new material or machine instead of guessing, built around four components used in sequence:

```
DoE Design --> (print + measure) --> RSM Evaluate --> Desirability
        ^
        |
Test Geometry (sample shapes for the print runs)
```

8.1 DoE Design

Generates a run plan for a chosen set of factors (parameter names, low/high bounds) and a design type: **DSD** (definitive screening design, 4 to 12 factors: good for a first pass to find which factors matter), **BBD** (Box to Behnken, 3 to 7 factors, 15/27/46/54/62 runs: supports a full quadratic response surface), **Taguchi** (L4/L8/L9/L12/L16/L18/L27 orthogonal arrays), **FullFact2** (full 2-level factorial, 1 to 10 factors), **FracFact2** (fractional factorial, 4 to 9 factors, named generator sets with a reported resolution).

Parameter	Description
`names`, `lows`, `highs`	Factor names and bounds.
`design`	Default `"BBD"`.
`responses`	Response names (creates empty columns to fill in).
`centres`, `centerPts`, `levels`, `seed`, `fracRuns`	Advanced group.
`sigma`, `effect`, `sims`, ...	Power group (2.3.0): Monte-Carlo power analysis: "can this 15-run plan detect a 0.3 mm effect" answered before any clay is mixed, with a warning under the conventional 0.8.
`xlsxPath`, `csvPath`	Export group.

Outputs: `runs` (actual-units tree), `coded` (-1 to $+1$ tree), `factorNames` (feeds `RSM Evaluate`), `runLabels`, `table`, `power` (when the power group is active), `info`.

8.2 Printing and measuring the runs

Use **Test Geometry** to generate a consistent sample coupon per run, slice/place/G-code it with the settings from that DoE row, print it, and measure the chosen responses by hand (e.g. dimen-

sional accuracy, surface finish score, collapse/overhang behaviour, cycle time). Right-click dropdown shapes: Cylinder, Vase, TwistedVase, Frustum, Wall, Box.

Parameter	Unit	Default	Description
`plane`			Placement group.
`height`	mm	100	Overall height.
`radius`	mm	40	Overall radius.
`waveAmp`, `waveCount`, `twistDeg`, `sides`, `topRatio`	mixed		Profile group (shape-dependent).
`length`, `width`	mm		Wall/box group.

Outputs: `geometry` (feeds `Slice`), `sections`, `info`.

8.3 RSM Evaluate [Heavy]

Fits a response-surface model (Linear / 2FI / Quadratic) per response via ordinary least squares (QR decomposition), reporting coefficients with standard error/t/p-values, R^2/adjusted-R^2/predicted-R^2, RMSE, and a full ANOVA including a lack-of-fit vs. pure-error split.

Parameter	Description
`tablePath`	Filled-in measurements file.
`factorNames`	From `DoE Design`.
`responseNames`	Auto-detected if left blank.
`model`	Linear / 2FI / Quadratic.
`lows`, `highs`, `centres`	Ranges group.
`X`, `Y`	Direct-data group (bypasses the file if data already lives in Grasshopper).
`xlsxReport`	Export group.

Outputs: `models` (feeds `Desirability`), `report`, `equations`, `predicted`, `json`, `diag` (2.3.0: leverage, studentized residuals, Cook's distance per run, VIF per term, with warnings on Cook's $D > 1$ / VIF > 10), `info`. **Watch for a weak-fit warning (adjusted-R^2 < 0.5)**: a weak model here means the downstream Desirability recommendation is only as good as a poor fit, not a reliable optimum.

File formats. `tablePath` accepts Excel or CSV. CSVs are detected per file as English (`1.5`) or German (`1,5`) number format; mixed conventions, thousands separators or malformed quoting raise a `CsvFormatException` naming the line, independent of the Windows locale: never a silently wrong number.

8.4 Desirability [Heavy]

Takes the fitted `models` plus a **goal string per response** (a small DSL: `max`, `min`, `target=7 in 6..8`, `range=L..H`, with optional weight `w=` and shape `s=` modifiers) and searches for the factor settings that best satisfy all goals simultaneously, using Derringer to Suich weighted geometric-mean desirability, a grid search (capped at 500,000 points) plus Nelder to Mead refinement for precision.

Parameter	Description
`models`	From `RSM Evaluate`.
`goals`	Goal DSL strings, one per response.
`factorPriors`	Priors group: biases the search toward preferred regions.
`lows`, `highs`, `gridPerDim`	Bounds group; `gridPerDim` default 15.
`sweepResponse`, `sweepValues`, `sweepBand`	Sweep group: reproduces a "recipe per target value" style campaign.
`xlsxPath`	Export group.

Outputs: `settings` (recommended actual-unit factor values), `D` (overall desirability, 0 to 1), `predicted`, `report`, `alternatives` (*local optima of the scalarised D*, not a Pareto set), `sweep`, `interval` (2.3.0: 95 % prediction/confidence intervals; alternatives whose predictions sit inside the interval of the recommendation are marked ≈`best` rather than ranked), and: with the optional **pareto** group enabled: `pareto` (a non-dominated front over the individual goal desirabilities, with a 2-objective scatter in the report; 3+ objectives get a note instead of a misleading 2D projection).

A Graph-Mapper-style **goal-curve strip** (prototype, 2.3.0) appears below the desirability profile after the first solve: drag the ramp ends, target, or shape handle instead of typing goal text. Edits are stored as per-response overrides of the wired goal string (never modifying it), survive save/reopen, and reset with the `X` control; weight stays text-only and there is no undo.

9 Utility components: Panel 1 Slice

9.1 Graph Mapper

"Remap numbers through an editable curve: drag the grips on the canvas." The stock Graph Mapper's job with improved mechanics: pick a curve family (including custom point curves you own), drag grips directly on the component, and feed the remapped stream into any numeric input: a flow or speed gradient over layers is the typical use. Styling follows the Grasshopper canvas skin.

10 Rationale: pressure and pump

10.1 Pump extrusion (MAI 2PUMP PICTOR-3D)

The `pump` group on `G-code Generator` computes the volumetric flow the pump needs to deliver:

$$Q = h \cdot w \cdot v \cdot \eta$$

(layer height × layer width × print speed × `slipFactor`), converted from mm^3/s to l/min, using `firstLayerSpeedFactor` for the first layer. Example from the test suite: $h = 10\,\mathrm{mm}$, $w = 30\,\mathrm{mm}$, $v = 50\,\mathrm{mm/s} \rightarrow$ **0.9 l/min**. `slipFactor` (default 1.0) is a calibration term back-calculated after a real print by comparing planned vs. actually deposited volume: typically ≥ 1.0.

This group does not touch the emitted G-code. The E-values in the file remain exactly the ram/plunger volumetric model's output, unchanged, regardless of whether the pump group is on. This is deliberate: the MAI pump is driven by its own 0 to 10 V analog speed interface, not the motion controller's E-axis, so there is nothing meaningful to write into the G-code without knowing the specific control wiring: leaving E intact is harmless if pump-driven and still correct if ram-driven. (If a controller does drive the pump: e.g. a Duet `M950`/`M42` PWM mapping: a flavor-specific control line could be added; it is not currently implemented.)

Calibration is measured, never hardcoded. `pumpCalibCsv` is a `setting, flow_lpm` table; the lookup is **piecewise-linear interpolation only**: it will not extrapolate beyond the measured range (a warning and no recommendation, rather than a guess, results outside that range). This is an explicit policy decision: manufacturer figures for this exact pump disagree across sources (mai.at currently lists 0.7 to 15.5 l/min; an older listing says 0.5 to 5.0 l/min; the lab's actual unit was confirmed by the user on 2026-07-06 as the 0.5 to 5.0 l/min variant): and even correct manufacturer numbers would not account for a specific material's rheology, pump wear, or hose setup. The shipped CSV is a placeholder full of `NEEDS_INPUT` markers.

Bucket-test calibration protocol (approximately 15 minutes).

1. Mix material and prime the line.
2. Choose 4 to 6 settings spanning the working range (e.g. 2, 4, 6, 8, 10 V).
3. Run each into a tared bucket for 60 seconds and weigh.
4. Measure fresh material density separately with a 1-litre cup.
5. Compute $flow_lpm = \frac{mass_g}{density_{g/ml} \times 1000 \times time_{min}}$.
6. Enter rows in ascending setting order in `pumpCalibCsv`.

Repeat whenever material/mix or hose length changes: this is not a one-time calibration.

10.2 Legacy calibration values (30 mm ram machine)

The only surviving *field-measured* calibration for a smaller, 30 mm-bore ram cartridge machine (as opposed to the current 94 mm-ram defaults), preserved from the deleted Python v1 toolkit's `RamGcodeBody.py`:

Parameter	30 mm-ram (legacy, measured)	94 mm-ram (current default)
`layerHeight`	3.02 mm	2.0 mm
`width`	7.0 mm	4.0 mm
`pressureCoeff`	1.3	1.0
`printSpeed`	43.0 mm/s	20.0 mm/s
`travelSpeed`	40.0 mm/s	60.0 mm/s
`retractSpeed`	10.0 mm/s	2.0 mm/s
`pressureOffset`	1.0 mm	0.1 mm
`zHop`	2.0 mm	0.0 mm

If printing on a 30 mm-ram machine, start from the legacy column: it was deliberately rescaled for the 94 mm defaults, so do not "correct" the current defaults back toward these unless the hardware actually matches the 30 mm case. Note this table reflects the historical Python prototype's defaults (`width` 4.0 mm, `printSpeed` 20 mm/s), which differ from the current defaults of the plugin listed in §5 (`width` 10.0 mm, `printSpeed` 43 mm/s): the defaults of the plugin have since been retuned independently of this legacy record.

10.3 A real historical bug worth knowing about (ArcWelder)

Until fixed on 2026-07-14, the ArcWelder CLI integration passed a path tolerance as a bare number (`5.0`) where the executable actually expects a fraction in $(0, 1)$ (`0.05` = 5%): the CLI rejected every call and the code **silently fell back to unwelded (plain G1) output** with no visible error. Any G-code generated by `arcwelder` from before this fix was never actually welded despite

reporting success; the fix now also surfaces the CLI's exit code and stderr on failure instead of silently succeeding.

11 Known limits and explicit placeholders

The following are shipped defaults that make the tool work out of the box, **not** calibrated values for any particular material or machine:

- `Overhang`'s `warnDeg`/`failDeg` (25°/45°).
- The pump group's shipped `pumpCalibCsv`: placeholder, `NEEDS_INPUT`.
- The `ecomp` group's `eCompRefMul` (1.0 = no-op): uncalibrated.

Other known limits:

- An **unwired** `robot` input on the G-code generator keeps the legacy Potterbot coordinate frame (mirror about $y = -x$) for backwards compatibility: wire a definition to get the identity frame a normal Cartesian machine expects. The active frame is always visible in `info`.
- Extrusion volume off-planar (adaptive/conical modes) is only nominal unless `varh` is enabled on the G-code component.
- Time estimates are a lower bound unless `Print Stats`'s `accel` input is set non-zero (opt-in trapezoidal model) or a kinematic robot definition is wired (joint-limited model).
- Unwired Duet control components (7 Print) default to the lab's Potterbot SCARA; the SCARA calibration components are SCARA-only.
- A known pre-existing bug in wall offsetting around hole orientation is logged but not yet fixed.

12 Troubleshooting

Symptom	Check
Heavy component shows stale/empty result	Click its compute button: it does not auto-recompute.
`spacing` throws a hard error	Must be > 0.
Volume/time implausible	Does `layerHeight` match between Slice and G-code Generator?
Conical/adaptive volume looks off	Enable the `varh` group on G-code Generator.
Robot Simulation shows a red limits error	Do not bypass: fix geometry/placement/speed, re-simulate.
G-code looks mirrored on a Cartesian machine	Wire a Robot Definition: unwired keeps the legacy Potterbot frame (check `frame:` in `info`).
Part lands somewhere odd after changing document units	Heavy caches are invalidated on a unit change: press Compute again; mm-labelled inputs stay physical mm.
RSM/pump CSV refused	The named line mixes English/German number formats (or has broken quoting): export one consistent convention.
Pump group changes nothing in G-code	Correct: pump only calculates a setting, never touches E.

Symptom	Check
ArcWelder output looks unwelded	Confirm the CLI actually ran (info/exit code): was a real historical bug, fixed 2026-07-14.
Spiralize falls back / fails	A layer has $\neq 1$ closed contour: simplify the profile.
Save G-code never writes	`write` needs a Button (one-shot), not a persistent Toggle.
Save reports "writing ..." but no `written = true`	Writes are asynchronous: the confirmation arrives on the follow-up solve; a failure surfaces as a warning instead.
RSM Evaluate fit looks unreliable	Check adjusted-R^2: below 0.5 is a flagged weak fit.

13 Testing and provenance

`Lugworm.Core.Tests` (980+ tests as of 2.6.0) runs the algorithmic core independent of Grasshopper/Rhino: arc fitting, offset/clipping, DoE design generation and inference, G-code parsing/restart, geometry validation, robot IK/limit checks (SCARA and Cartesian gates), machine-fingerprint trust, CSV format detection, SemVer/update check, and the pump/variable-flow/slope-compensation math each have dedicated test files. If `Lugworm.Core` is modified, run this suite before trusting new behaviour in Grasshopper.

`docs/DECISION_LOG.md` in the project root records every non-obvious design decision (the ambiguity, the choice, the reason, and how to change it) across all development phases: consult it before assuming a behaviour is a bug rather than a deliberate, documented choice. `docs/AUDIT_PHASE0*.md` and `docs/RUN_REPORT*.md` record point-in-time snapshots of what was fixed or added in each development pass, including the ArcWelder and robot-fingerprint fixes described in section 8.3 and section 2.1.